Sistema de Gestión del Comité de Investigación

Explicación de la lógica del código

Guía para el equipo: qué hace cada parte, cómo funciona el flujo de una solicitud y cómo responder las preguntas más probables

Contenido
1. Qué es y con qué está hecho · 2. Cómo ejecutarlo (MySQL Server + Workbench)
3. Arquitectura · 4. Base de datos · 5. Roles y permisos
6. Flujo de una solicitud · 7. Trazabilidad y notificaciones · 8. Fechas y vencidas
9. Dashboard y reportes · 10. Preguntas probables · 11. Puntos débiles y mejoras

1. Qué es y con qué está hecho

Es una aplicación web que gestiona las solicitudes que los investigadores presentan al Comité de Investigación, desde que se **radican** hasta que se **deciden** y quedan registradas en un **acta**. Todo queda con trazabilidad: quién hizo qué y cuándo.

Tecnología	Para qué se usa
Python + Flask	Servidor web y rutas (URLs).
Jinja2	Plantillas HTML en la carpeta `templates/`.
MySQL Server	Base de datos `comite_db`. Se conecta con `mysql-connector-python`.
MySQL Workbench	Herramienta visual para ver las tablas y los datos (no la usa el código).
CSS propio	`static/style.css`.

2. Cómo ejecutarlo (MySQL Server + Workbench)

Workbench es solo el cliente visual; quien guarda los datos es **MySQL Server** (normalmente se instala junto con Workbench). La aplicación se conecta directamente al servidor, así que funciona con cualquier instalación de MySQL.

1. Verificar que el servicio **MySQL Server** esté corriendo (`services.msc` → `MySQL80`, o en Workbench: *Server* → *Startup/Shutdown*).
2. Instalar dependencias: `python -m pip install -r requirements.txt`
3. Indicar la contraseña de `root` (la que se puso al instalar MySQL). Por defecto el código usa `root` **sin contraseña**.
4. (Opcional) Cargar datos de ejemplo: `python datos_demo.py`
5. Iniciar: `python main.py` y abrir `http://127.0.0.1:5000`

```
$env:MYSQL_PASSWORD = "contraseña de root"
python datos_demo.py
python main.py
```

Al arrancar, la aplicación **crea sola** la base `comite_db` y todas las tablas. No hay que importar ningún archivo SQL. En Workbench aparece en *Schemas* (refrescar si no se ve).

Rol	Correo	Contraseña
Administrador	admin@comite.edu.co	admin123
Secretario	secretaria@comite.edu.co	secre123
Comité	comite@comite.edu.co	comite123
Investigador	investigador@comite.edu.co	invest123

3. Arquitectura: tres piezas

Archivo	Responsabilidad
`main.py`	**Controlador.** Recibe las peticiones HTTP, verifica permisos, lee los formularios y llama a `models`. No contiene SQL.
`models.py`	**Modelo y lógica de negocio.** Crea la base de datos, contiene todo el SQL, las reglas del flujo, las notificaciones y el historial.
`templates/`	**Vistas.** Muestran los datos. `base.html` es la plantilla padre (menú y campana de notificaciones); las demás la heredan.
`datos_demo.py`	Carga un escenario de ejemplo usando las mismas funciones del sistema. Se puede ejecutar varias veces sin duplicar datos.

Es un patrón tipo **MVC**: `main.py` decide **quién puede hacer qué** y `models.py` decide **qué pasa cuando se hace**.

4. Base de datos

Al ejecutar `python main.py` se llama a `init_db()` (`models.py:385`), que:

1. Crea la base `comite_db` si no existe.
2. Crea las tablas con `CREATE TABLE IF NOT EXISTS`.
3. Aplica **migraciones**: si la base venía de una versión anterior, agrega las columnas que falten consultando `information_schema`.
4. Inserta los 4 roles, los 5 tipos de solicitud con sus requisitos y los usuarios demo.

Tablas principales

Tabla(s)	Qué guarda
`usuario, rol, investigador`	Usuarios y su rol. El investigador es un usuario con datos extra (documento, programa, facultad, teléfono).
`tipo_solicitud, requisito`	Cada tipo exige ciertos documentos, obligatorios o no.
`solicitud`	Tabla central: código, título, estado, prioridad, fecha límite.
`documento`	Archivos subidos, con número de versión y requisito asociado.
`validacion_requisito`	Si cada requisito cumple o no, con observación.
`asignacion_revision, revision`	A quién se asignó la revisión (con fecha límite) y el concepto que dio.
`observacion`	Comentarios generales y solicitudes de ajuste.
`sesion_comite, agenda_sesion`	Sesiones del comité y su orden del día.
`decision, acta`	Decisión por solicitud en cada sesión y el acta generada.

Tabla(s)	Qué guarda
`historial_estado, actuacion`	Trazabilidad: cambios de estado y línea de tiempo de acciones.
`notificacion`	Avisos para cada usuario, con enlace.

Conexiones y transacciones

- Se usa un **pool de 10 conexiones** reutilizables (`get_conn()`), lo que hace más rápida cada página.
- Todo pasa por el context manager `db()`: si todo sale bien hace `commit`, si ocurre un error hace `rollback`. Cada operación es **atómica**: cambiar el estado, guardar el historial y notificar ocurren todos o ninguno.
- Todas las consultas son **parametrizadas** (`?` → `%s`), por lo que no hay inyección SQL.

5. Roles y permisos

Hay 4 roles: **Investigador, Comité, Secretario y Administrador**. En `models.py` se agrupan:

- `ROLES_GESTION` = Administrador + Secretario (gestionan el trámite).
- `ROLES_COMITE` = ROLES_GESTION + Comité (pueden ver todo).

En `main.py` hay **decoradores** que protegen cada ruta:

- `@login_required`: si no hay sesión iniciada, redirige al login.
- `@roles_required(...)`: si el rol no está permitido, responde **403**.
- `@admin_required`, `@gestion_required`, `@comite_required`: atajos de lo anterior.

Punto clave: el investigador **solo ve sus propias solicitudes**. `_id_si_investigador()` devuelve su id y `obtener_solicitud` agrega `AND i.id_usuario = ?` a la consulta. Si un investigador intenta abrir la URL de una solicitud ajena, recibe **404**. Lo mismo aplica a la descarga de documentos.

Acción	Investigador	Comité	Secretario	Admin
Radicar y cargar documentos de sus solicitudes	Sí			
Ver todas las solicitudes		Sí	Sí	Sí
Validar requisitos, asignar responsables, cambiar estado			Sí	Sí
Emitir concepto de revisión		Si está asignado	Sí	Sí
Solicitar ajustes		Sí	Sí	Sí
Sesiones, agenda, decisiones y actas		Consulta	Sí	Sí
Reportes		Sí	Sí	Sí
Administrar usuarios				Sí

6. Flujo de una solicitud (lo más importante)

```
Radicación → Recibida
   ↓  (secretario valida requisitos)
   ├─ falta algo obligatorio → Pendiente por documentos
   │                          → investigador carga y reenvía → Recibida
   └─ todo cumple → En revisión
         ↓  se asignan revisores → cada uno emite un concepto
         ├─ alguien pide ajustes → Requiere ajustes
         │                         → investigador reenvía → En revisión
         ↓  el secretario la marca
      Lista para comité → se agenda en una sesión → Decisión
         ├─ Aprobada / Aprobada con ajustes / Rechazada
         └─ Aplazada → vuelve a Lista para comité
      → Acta → Cerrada
```

#	Paso	Función	Qué hace
1	**Radicación**	`crear_solicitud`	Solo la hace un investigador. Se genera un código consecutivo **SOL-AAAA-NNNN** (`_nuevo_codigo`), queda en estado *Recibida*, se registra en el historial y se notifica a secretaría y administración.
2	**Carga de documentos**	`guardar_documento`	Valida la extensión con una lista blanca (pdf, docx, xlsx, etc.). Guarda el archivo en `uploads/<id_solicitud>/` con un prefijo aleatorio (uuid) para que no se sobrescriban nombres. Si se sube otro archivo para el mismo requisito, aumenta la **versión**. Límite: 16 MB.
3	**Validación de requisitos**	`validar_requisitos`	El secretario marca «cumple» o no en cada requisito. Si falta **algún obligatorio** → *Pendiente por documentos*; si todo cumple → *En revisión*.
4	**Asignación de responsables**	`asignar_revisor`	Se elige un integrante del comité y una fecha límite. Recibe notificación. No se puede asignar dos veces la misma solicitud a alguien que ya tiene una revisión pendiente.
5	**Revisión**	`registrar_revision`	El revisor da un concepto (*Favorable*, *Favorable con ajustes* o *Desfavorable*) y su asignación pasa a *Completada*.
6	**Ajustes**	`agregar_observacion` / `responder_ajustes`	Una observación de tipo «Solicitud de ajuste» pasa la solicitud a *Requiere ajustes*. El investigador responde y vuelve a *En revisión*.
7	**Lista para comité**	`cambiar_estado`	El secretario cambia el estado manualmente, indicando un motivo.
8	**Sesión y agenda**	`agregar_a_agenda` / `consolidar_agenda`	Se crea la sesión y se agregan solicitudes una por una, o con **Consolidar**, que agenda de una vez todas las que están en *Lista para comité*, ordenadas por prioridad.
9	**Decisión**	`registrar_decision`	El diccionario `DECISIONES` convierte cada decisión en el nuevo estado. «Aplazada» la devuelve a *Lista para comité*.

#	Paso	Función	Qué hace
10	**Acta**	`generar_acta`	**No se puede generar si alguna solicitud de la agenda no tiene decisión.** El texto se arma automáticamente (asistentes, orden del día, decisiones con conceptos, resumen y firmas). Si se vuelve a generar, la versión aumenta. La sesión queda como *Realizada*.

¿Cuándo puede editar el investigador? Solo mientras la solicitud está en *Recibida*, *Pendiente por documentos* o *Requiere ajustes* (`ESTADOS_EDITABLES`). Después ya está en trámite y queda bloqueada.

7. Trazabilidad y notificaciones

Todo cambio de estado pasa por **una sola función**, `_cambiar_estado`, que:

1. Actualiza el estado de la solicitud.
2. Guarda el cambio en `historial_estado` (estado anterior → nuevo, quién y motivo).
3. Registra una `actuacion` (la línea de tiempo).
4. Notifica al investigador.

Al haber una sola función, **ningún cambio de estado queda sin registrar**. Las notificaciones son filas en la tabla `notificacion` con un enlace a la solicitud. La campana del menú muestra cuántas no se han leído gracias a un `context_processor` que inyecta ese dato en todas las páginas.

8. Fechas y solicitudes vencidas

Una solicitud está **vencida** si tiene fecha límite, esa fecha ya pasó y **no está en un estado final** (Aprobada, Aprobada con ajustes, Rechazada, Cerrada). Lo calcula `_marcar_vencida`. Las revisiones asignadas siguen la misma lógica. Las vencidas se resaltan en la lista de solicitudes, el dashboard y los reportes.

9. Dashboard y reportes

- **Dashboard:** conteo por estado, vencidas, mis revisiones pendientes, próximas sesiones y actividad reciente. Para el investigador todo se filtra a sus propias solicitudes.
- **Reportes** (`reportes()`): se calculan con SQL usando `GROUP BY` y se pueden filtrar por rango de fechas: por tipo, mes, programa y decisión; **tiempo promedio** desde la radicación hasta la primera decisión que no sea «Aplazada» (`DATEDIFF`); y carga de trabajo por revisor.
- **Exportar CSV** con separador `;` y BOM UTF-8, para que Excel abra bien las tildes.

10. Preguntas probables y respuestas cortas

¿Cómo se guardan las contraseñas?

Con un hash SHA-256 (función `hp()`); nunca en texto plano.

¿Qué pasa si algo falla a mitad de una operación?

El `rollback` en `db()` deshace todo; no quedan datos a medias.

¿Cómo evitan que un investigador vea solicitudes ajenas?

La consulta filtra por su `id_usuario`; si la solicitud no es suya, recibe 404.

¿Cómo se sabe quién hizo qué?

Las tablas `actuacion` e `historial_estado` guardan usuario y fecha de cada acción.

¿Por qué no se puede borrar una sesión?

Si ya tiene decisiones o acta, se bloquea para no perder la trazabilidad; se marca como *Cancelada*.

¿Cómo se protege la subida de archivos?

`secure_filename`, lista blanca de extensiones, límite de 16 MB y descarga solo con permiso.

¿Por qué el login revisa que `next` empiece con «/»?

Para evitar redirecciones a sitios externos (*open redirect*).

¿Dónde se crea la base de datos?

En `init_db()` al iniciar `main.py`; no hay que importar SQL.

¿Se puede usar con XAMPP o con Workbench?

Con cualquiera: el código solo necesita un servidor MySQL. Workbench es la herramienta visual para consultarlo.

¿Cómo se genera el código de la solicitud?

`_nuevo_codigo` busca el último consecutivo del año y suma 1: `SOL-2026-0001`, `SOL-2026-0002`…

11. Puntos débiles y posibles mejoras

Conviene conocerlos por si los señalan en la evaluación:

- **SHA-256 sin sal** no es lo ideal para contraseñas; lo recomendado es `werkzeug.security.generate_password_hash` (scrypt o pbkdf2).
- La `SECRET_KEY` tiene un valor por defecto de desarrollo y la app corre con `debug=True`. Sirve para la demo, no para producción.
- No hay protección **CSRF** en los formularios (se podría agregar con Flask-WTF).
- *Lista para comité* y *Cerrada* se asignan a mano: no hay una regla automática que, por ejemplo, pase la solicitud a *Lista para comité* cuando todos los revisores terminan.